

TinyML User Guide (Draft)

UG-TinyML

V2.0



易灵思（深圳）科技有限公司

版权所有，侵权必究

2024/5/20

修订记录

日期	修订版本	描述	作者
2024-5-20	V1.0	添加 EVSOC 模块概要设计	
2024-5-20	V2.0	更新工程部署和量化及新功能描述	

目录

1	产品特性简介	1
2	设计框架	1
2.1	Sapphire RISC-V SoC	3
2.2	DMA Controller.....	3
2.3	HyperRAM Controller	3
2.4	Video Capture and Pre-Processing	4
2.5	Post-Processing and Display	4
2.6	Hardware Accelerator	4
2.7	TinyML Accelerator.....	5
2.8	固件选择	6
2.8.1	Hardware Accelerator	6
2.8.2	TinyML Accelerator.....	6
2.9	使用您自己的加速器	7
2.9.1	接口修改	7
2.9.2	DMA Controller 修改.....	8
2.9.3	FIFO 修改.....	8
2.9.4	RISC-V 修改	8
3	环境搭建	8
3.1	运行环境安装	8
3.1.1	安装 Efinity®软件	8
3.1.2	Install the Edge Vision SoC	9
3.1.3	Install the RISC-V SDK	9
3.1.4	Install the Java JRE	9
3.2	硬件安装	9
3.2.1	硬件组件需求	9
3.2.2	硬件组装步骤	9

3.2.3	安装 USB 驱动.....	10
3.2.4	下载程序到开发板	10
3.3	RISC-V SDK	10
3.4	创建 TinyML 工程	10
3.4.1	TinyML 开发流程和工程结构	10
3.4.2	Tflite 模型导入.....	12
3.4.3	TensorFlow 训练后量化	27

图目录

未找到图形项目表。

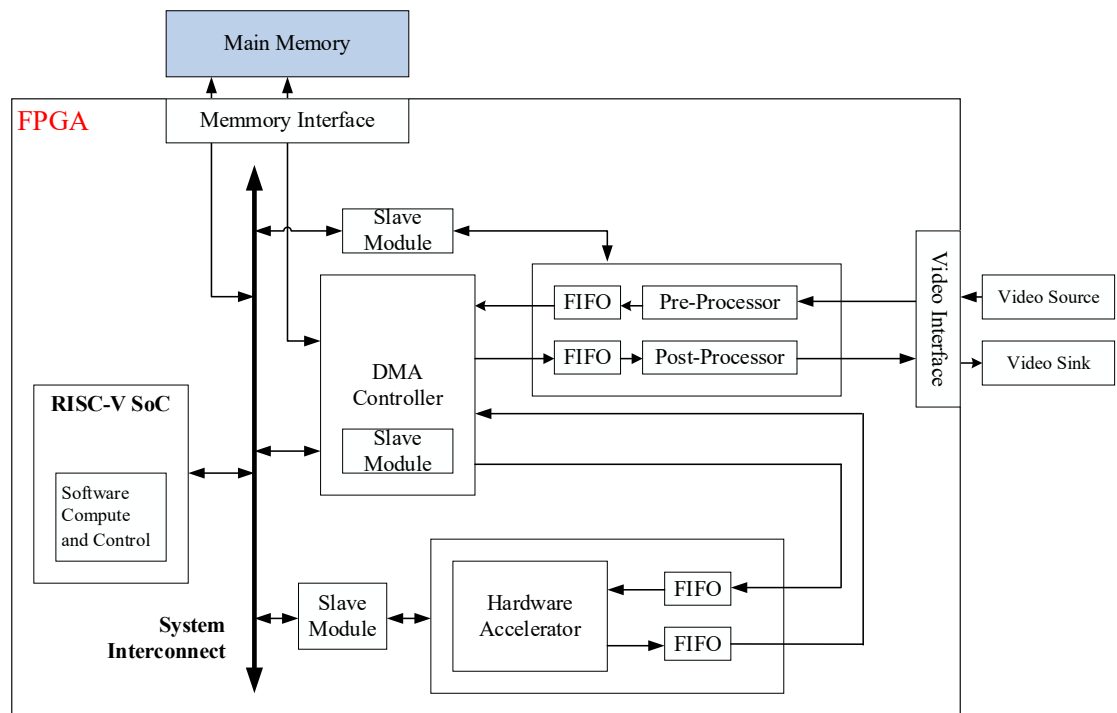
表目录

未找到图形项目表。

1 产品特性简介

人工智能（AI）和边缘视觉是一个新兴领域，涵盖了汽车、工业视觉、零售和机器人等广泛的应用。易灵思的 Edge Vision SoC 框架使用 RISC-V SoC 软核和硬件相结合的方式来实现，包含了特定的嵌入式软件功能、硬件加速器模块、AI 算法加速器模块、存储和 I/O 相关接口、外围设备和控制逻辑。其 Edge Vision SoC 框架如图所示。

在此框架内，易灵思选择为某些特定功能提供了示例设计，例如视频处理、AI 对象检测、机器学习、多摄像机融合等。你可以自由地使用 Edge Vision SoC 框架编写 C/C++ 代码，构建不同的加速算法或者 AI 模型，或者你也可以完全使用自己的定制插件。



本用户指南主要描述了如何使用以 Edge Vision SoC 框架构建的示例设计。这些设计说明了本框架的用例，特别是视频处理的硬件/软件协同处理设计。此外，本设计还显示了如何使用软件控制 FPGA 硬件，也就是说，您可以通过更改 RISC-V 处理器中的固件来启用不同的加速功能。

2 设计框架

如下图，针对易灵思 Titanium Ti60 F225 开发板所设计的图像信号处理实例设计框图如下。本次设计主要依靠易灵思的 Edge Vision SoC 框架，在本次设计中添加了 TinyML Accelerator，主要用于对视频流的 AI 加速算法。

DMA Controller	6162	745	8,393	67	0
CSI-2 RX Controller Core	871	60	2149	15	0
DSI TX Controller Core	1574	111	4506	21	0
HyperRAM Controller	1738	263	2601	18	0
Camera	814	618	1039	11	0
Display	489	0	716	12	0
Hardware Accelerator	630	162	606	8	0
TinyML Accelerator					

2.1 Sapphire RISC-V SoC

本设计实例支持高度灵活的硬件和软件的协同设计方法，因此您可以选择是在 RISC-V 处理器中执行算法运算，还是在硬件中执行算法。RISC-V SoC 可用于执行整体系统控制并执行固定的或者灵活多变的算法。

在本设计实例的框架中，其包含的 Sapphire Vision RISC-V SoC 是 Sapphire SoC 标准版本的变体。详情请参阅《Sapphire RISC-V SoC Data Sheet》规格数据手册。

2.2 DMA Controller

DMA 控制器主要实现便于外部存储器和本设计中的其他模块之间的通信功能。实现了将数据帧存储到外部存储器中、向 Hardware Acceleration 发送和接收数据，并向 Post-Processing 发送数据。DMA 控制器对每个端口都有专用的、固定方向的通道。

DMA 控制器有 direct 模式和 scatter and gather (SG) 模式。在这两种模式中，RISC-V 处理器使用轮询(默认)或中断的方式来原因 DMA 传输是否完成。

- 轮询——RISC-V 处理器检查 BUSY 信号，直到其被取消断言。
- 中断——RISC-V 处理器可以在 DMA 传输过程中执行其他操作，当它接收到一个中断时代表传输完成。

在本次 RTL 设计中，DMA 控制器连接到外部 256 位的 DDR DRAM 接口。

2.3 HyperRAM Controller

Hyper RAM 控制器主要实现了对片外存储芯片 (Hyper RAM) 的读写管理, 其一端为 AXI4 接口, 一端为 Hyper RAM 的管脚。可以支持三个 master 端同时对 Hyper RAM 的读写操作。

2.4 Video Capture and Pre-Processing

本次设计使用 Raspberry Pi V2 相机模块作为系统图像数据的输入来源。视频从摄像机传输到 MIPI CSY-2 RX 控制器内核 (Titanium), 分辨率为 1920*1080, RAW10 格式 (每个原始像素 10 位), 每个时钟 4 个像素。帧速率根据相机驱动程序设置而变化。

Pre-Preprocessor 接收来自相机的视频流并执行以下功能:

- 根据您在 RISC-V 固件中选择的设置调整 RGB 增益。
- 将每个时钟的像素数从 4 更改为 2。
- 将 RAW 转换为 RGB。
- 裁剪或缩放视频以进行后续处理。
- 可选择将 RGB 像素数据转换为灰度像素数据。

对于 Titanium Ti60 F225 开发板设计, 输入视频分辨率被裁剪为 1080*1080, 然后使用 nearest-neighbor 法缩小到 540*540。

完成这些功能后, Pre-Preprocessor 通过 DMA 控制器将数据存储到外部存储器中。如果 RISC-V 处理器上运行的固件指定将 RGB 或灰度数据存储在外存储器中。此时您可以选择在硬件中实现灰度转换, 或者在 RISC-V 处理器中执行该功能。

2.5 Post-Processing and Display

Post-Processor 根据最终的显示器分辨率生成视频信号 (HSYNC、VSYNC 和 VALID), 并从显示器 DMA FIFO 检索像素数据 (红色、蓝色和绿色)。对于 Trion 开发板设计, 视频信号和像素数据通过 LVDS 接口打包和输出。对于 Titanium Ti60 F225 开发板设计, 视频信号和像素数据被传递到 DSI TX 控制器内核, 便于在 Mini-DSI 面板上显示。

2.6 Hardware Accelerator

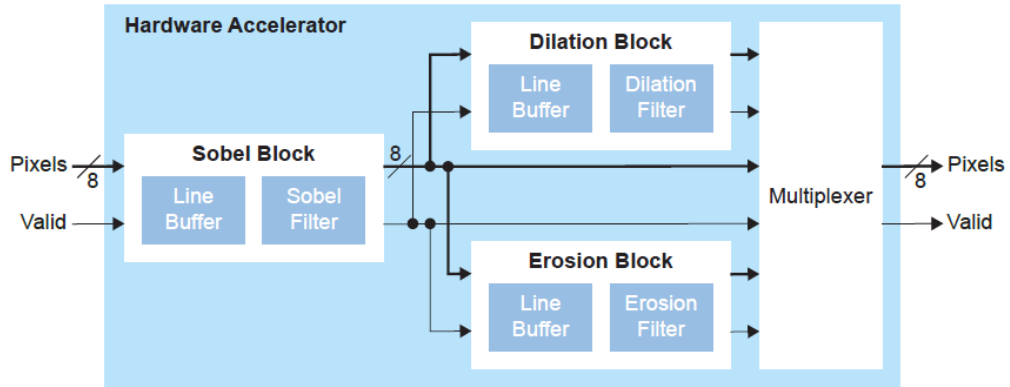
本次设计在硬件加速器中提供了几个标准功能, 这些功能也可以使用软件代码来实现。在默认情况下, 本设计演示了硬件/软件协同设计: RISC-V 处理器作为嵌入式软件执行 RGB 到灰度的转换, 而硬件加速器执行 Sobel 边缘检测、二进制腐蚀 (binary erosion) 和二进制膨胀 (binary dilation)。

- Sobel filter: 执行边缘检测。
- Binary erosion: 通过对所有窗口像素进行“与”运算来删除线条细节。

- Binary dilation: 通过对所有窗口像素进行“或”运算来增强线条细节。

硬件加速器模块在流水线流式体系结构中实现,并以三种模式运行:Sobel、扩展 Sobel 或侵蚀 Sobel。您可以在 RISC-V 处理器上运行的固件中指定模式。

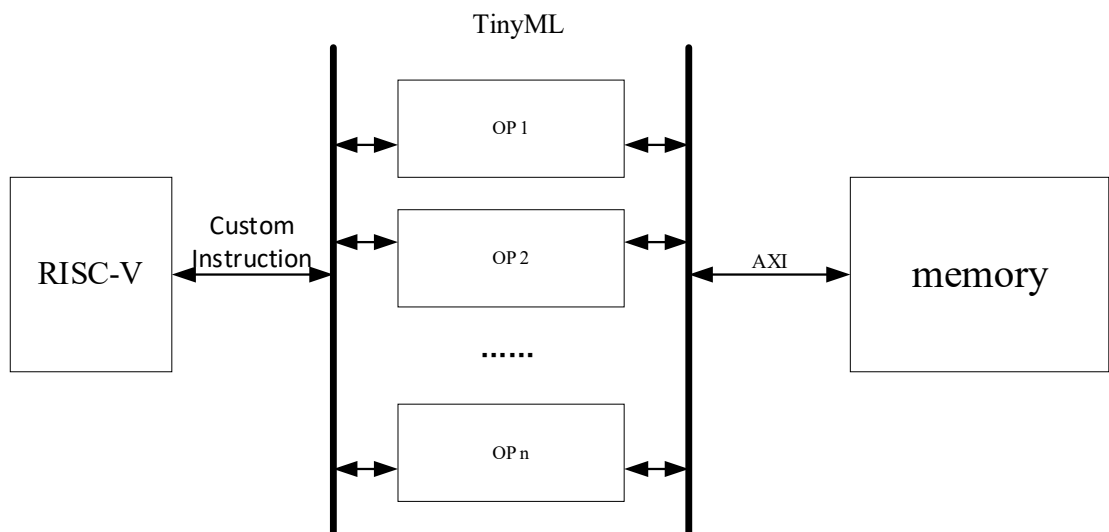
Figure 9: Hardware Acceleration Block Diagram



在 RTL 设计中,标准包装器包含加速功能、输入和输出 FIFO,以及与连接到 RISC-V 处理器的 AXI4 从机模块接口的调试和控制寄存器。FIFO 和控制逻辑从外部存储器获取输入数据,并通过 DMA 控制器将输出数据存储到外部存储器。此包装器旨在提供一个标准接口,以便您可以轻松地放入自己的加速函数。

2.7 TinyML Accelerator

微型机器学习 Tiny Machine Learning (TinyML) 是机器学习的一个研究领域,专注于在超低功耗的微控制器设备上开发和部署机器学习模型。TinyML 使机器学习可以在安全、低延迟、低功耗和低带宽的边缘设备上运行。本次设计提供了几个在 TinyML 上的实际加速应用,例如 `face_landmark`、`yolo_person`、`resnet_image_classification`、`mobilenetv1_person_detection` 和 `ds_cnn_keyword_spotting`。你可以灵活地选择自己所需要的加速应用。



在 RTL 设计中，TinyML 加速器主要是根据这些加速应用在软件上耗时较长的操作转变而来，将其由软件实现变为硬件实现以此加速。故 TinyML 加速器主要由几个加速操作模块（OP）构成，其一端接着 RISC-V 处理器的 custom instruction 接口，一端使用 AXI4 接口与外部的存储空间 Hyper RAM 相连。你也可以轻松的使用这些标准接口来加入自己的加速算法。

在本设计中，主要实现了以下几个加速操作：分别为 Convolution & Depthwise、ADD、MAXIMUM（比较最大最小）、MULT 和 FC（Fully Connected）。可以通过修改对应的参数使能来打开或者关闭这些功能（注：当你在软件上关掉此功能时，在 RTL 代码中应该也关掉该功能），每一个操作模块内又有单独的参数可以来调整模块的性能与资源使用情况，以此来适应你的设计。

2.8 固件选择

当你将基本的视频流框架搭建好以后，就可以开始定制软件了。在 RISC-V 处理器上可以选择或者定制自己将要实现的功能。在本设计中提供了两个加速模块以供你使用，分别为 TinyML Accelerator 和 Hardware Accelerator。你可以在软件代码中的 main.c 文件里去使用它。该文件中包含了注释代码，您可以打开它使其执行各种功能。这些功能可以再软件里实现或者使用硬件实现。在你修改 main.c 文件以后，则必须重新编译软件并将新固件文件(.elf)下载到主板。下面将介绍本次设计中的一些预设功能。

2.8.1 Hardware Accelerator

此节将叙述如何在开发板上运行不同的固件。在硬件加速器模块里定义了三个不同功能的固件，你可以通过修改软件主程序（main.c）来选择运行的固件，具体实现方式可参考《edge-vision-soc-ug-v3.0.pdf》。

2.8.2 TinyML Accelerator

对于 TinyML 加速器的固件选择，在 RTL 设计上，只需要修改文件 define.v 里的参数就可以了。其中具体的参数含义可以参考 TinyML 模块接口信号定义。

```
define AXI_DW 128
define CONV_DEPTH_MODE "STANDARD"
define CONV_DEPTH_STD_IN_PARALLEL 8
define CONV_DEPTH_STD_OUT_PARALLEL 4
define CONV_DEPTH_STD_OUT_CH_FIFO_A 1404
define CONV_DEPTH_STD_FILTER_FIFO_A 375
define CONV_DEPTH_STD_CNT_DTH 1404
define CONV_DEPTH_LITE_PARALLEL 4
define CONV_DEPTH_LITE_AW 3
define ADD_MODE "STANDARD"
define FC_MODE "LITE"
define FC_MAX_IN_NODE 64
define FC_MAX_OUT_NODE 12
define MULT_MODE "LITE"
define MIN_MAX_MODE "LITE"
```

而在软件程序上，你需要修改 `define.h` 文件里参数，使得软件和硬件使用的功能能够对应上。或者你也可以使用易灵思提供的工具(`tinymml_参数生成器`)来生成两个 `define` 文件，然后再重新编译工程。

2.9 使用您自己的加速器

在本实例中有一个执行过滤功能的硬件加速器 (Hardware Accelerator) 和执行 AI 算法的加速器 (TinyML Accelerator)。

其硬件加速器的顶层文件里(`hw_accel_wrapper.v`)包含了硬件加速器、FIFO、一些控制逻辑和调试寄存器。本顶层具有连接到 DMA Controller 和 AXI4 slave 的接口。

AXI4 slave 接口用于 RISC-V 处理器和硬件加速器之间的通信，主要用于调试和控制的功能。虽然本设计使用了 AXI4 slave 接口，但您也可以在顶层中更改为使用其它 slave 接口（如 APB3 或 AXI4-lite）。

顶层文件中的控制逻辑主要是为了产生 FIFO 和 DMA 读/写相关信号，这些读写信号根据硬件加速器的行为（例如，对数据流的读写请求、DMA 传输的固件设置等）来产生。

AI 算法的加速器的顶层文件里 (`tinymml_accelerator.v`) 拥有 AXI4 master 接口和 custom instruction 接口，其中 AXI4 接口是用来连接 DMA 控制器的，以便于从存储空间里读写数据；而 custom instruction 则是用来连接 RISC-V 处理器，可以接收相关参数，命令和数据等。

如果您想加入自己的自定义加速器，则需要对顶层文件和 RISC-V 的固件进行调整。以下各节介绍了您应该考虑的方面。

2.9.1 接口修改

首先决定要使用哪种类型的接口，例如 AXI4、APB3 或 AXI4-lite。接下来，确定 RISC-V 处理器和加速器之间需要什么通信来对控制和调试寄存器进行设置。

例如，您可以使用 RISC-V 处理器触发计算和握手的开始，设置不同的操作模式，或探测关键信号以进行调试。最后，根据您的设计更新的加速器顶层中的控制逻辑和调试寄存器。

2.9.2 DMA Controller 修改

第一步是确定输入和输出数据要求。DMA 控制器使用 AXI-ST 接口连接到硬件加速器。默认情况下，该接口具有 32bit 宽度，可用于从片外存储器中读写数据流。

DMA 控制器支持各个通道的异步模式。如果您需要以不同的时钟速率读写数据流，请将相应 DMA 控制器通道的时钟信号连接正确。（参见第 33 页的 DMA 控制器。）

2.9.3 FIFO 修改

根据自定义加速器的需要来确定 FIFO 要求（数据宽度、深度、模式等）和读/写行为，并相应地修改 FIFO。接下来，检查 FIFO 下溢和上溢，并相应地调整 FIFO 控制信号或 DMA 传输设置。

2.9.4 RISC-V 修改

当你修改了本设计的 RTL 代码后，您必须更新 RISC-V 固件以匹配您的 RTL 更改。例如，当地址偏移量增加后则需要去修改 `axi4_hw_accel.h` 文件中的内容。

如果您使用不同的 slave 接口，请在 `main.c` 中添加相应的标题以便于访问。请参阅 `main.c` 中的 Camera Capture、HW Accelerator、Check AXI4 slave Status（HW Accessor）和 Check APB3 slave States（Camera&Display）部分的 APB3 和 AXI4 从机接口的使用。

RISC-V Soc 使用易灵思的 IP 生成，可以在生成时选择自己需要的接口和运行频率，当使用自定义加速器时，应更新固件中的 DMA 传输设置。例如，根据加速器的行为更新每个捕获帧的 DMA 传输长度。（`dmag_direct_start()` 的第三个参数是以字节数表示的传输长度。）默认情况下，示例设计使用 `direct` 模式 DMA 传输。

3 环境搭建

3.1 运行环境安装

3.1.1 安装 Efinity®软件

如果从未使用过我们的产品，请先从支持中心下载 Efinity®软件并进行安装。要编译本用户指南中所描述的设计，您必须拥有 [Efinity IDE](#)

[v2023.2.307.2.307](#) 或更高版本的 Efinity®软件（注意：请不要在安装路径里使用中文或者空格）。

3.1.2 Install the Edge Vision SoC

3.1.3 Install the RISC-V SDK

3.1.4 Install the Java JRE

请执行以下操作安装 JRE：

1. 从下载适用于您的操作系统的 64 位版本的 JRE 或 JDK

<https://www.java.com/en/download/manual.jsp>（Java 8 官方版本）

<https://developers.redhat.com/products/openjdk/download>（OpenJDK 8 或 11）

<http://jdk.java.net/16/>（OpenJDK 16）

2. 按照网站上的安装说明安装 JRE。

3.2 硬件安装

3.2.1 硬件组件需求

本示例设计使用了 Titanium Ti60 F225 开发板工具包中的以下硬件：

- Titanium Ti60 F225 Development Board
- Mini-DSI 面板连接器子卡 * 1
- 双 MIPI 到 DSI 转换器子卡* 1
- Raspberry Pi 相机接口子卡*1
- Raspberry Pi v2 摄像头模块* 1
- Mini-DSI 面板* 1
- 15 针扁平电缆*1
- 30 针扁平电缆*1
- USB type-C 电缆*1
- 通用 AC-DC 电源适配器
- 跳线

您还需要自己提供以下硬件：

- 安装了 Efinity®软件的计算机

3.2.2 硬件组装步骤

硬件安装步骤请参考 [《Ti60F225 EvSoc》](#) 第 13 页。

3.2.3 安装 USB 驱动

3.2.4 下载程序到开发板

要使用 Edge Vision SoC 框架的示例设计，必须将 RTL 设计编程到板中。

1. 生成 Edge Vision SoC 框架的 bit 文件有两种方法：
 - 打开要使用的工程 (.xml)，查看并在 Efinity®软件中编译生成。
 - 使用位于 efinity_project 目录中已编译好的 edge_vision_soc.hex 文件。

注：提供的 bit 流文件是为 SPI Active 模式编译的。有关为其他模式生成 bit 文件的详细信息，请参阅《Efinity 软件用户指南》。

2. 使用 Efinity®Programmer 和 SPI Active 模式将比特流文件下载到开发板上。如果使用 SPI Active 模式，则您需要复位 FPGA。
3. 按下开发板上的 CRESET 按钮来复位 FPGA。此复位可确保在用户应用程序运行之前进行外部内存初始化。

3.3 RISC-V SDK

您可以使用 RISC-V SDK 来管理软件工程和进行调试。

Eclipse 软件使用步骤和方法请参考《[Elitestek RISC-V SDK](#)》

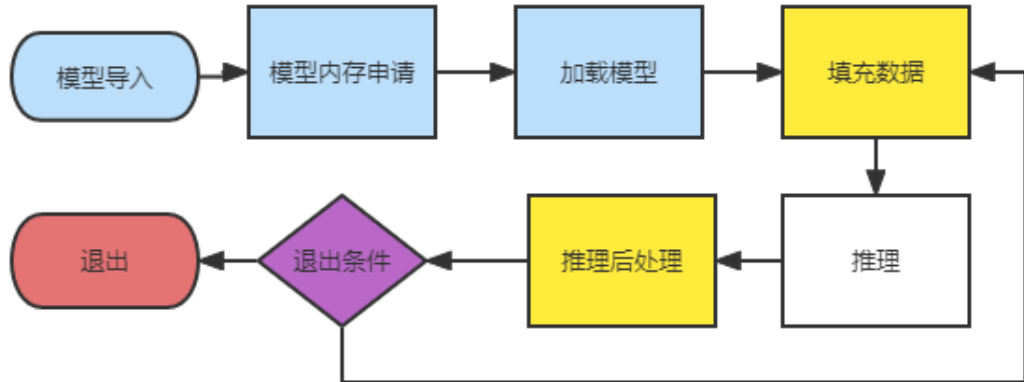
3.4 创建 TinyML 工程

TinyML 示例工程已经在附件中包含，您可以使用 Risc-V SDK 进行导入，下面是一份文件目录：

文件	说明
tools	Tinyml 模型解析和参数化工具
tinyml_vision	示例工程
model_zoo	模型训练示例
Ti60F225 开发板	用于执行 TinyML 工程

3.4.1 TinyML 开发流程和工程结构

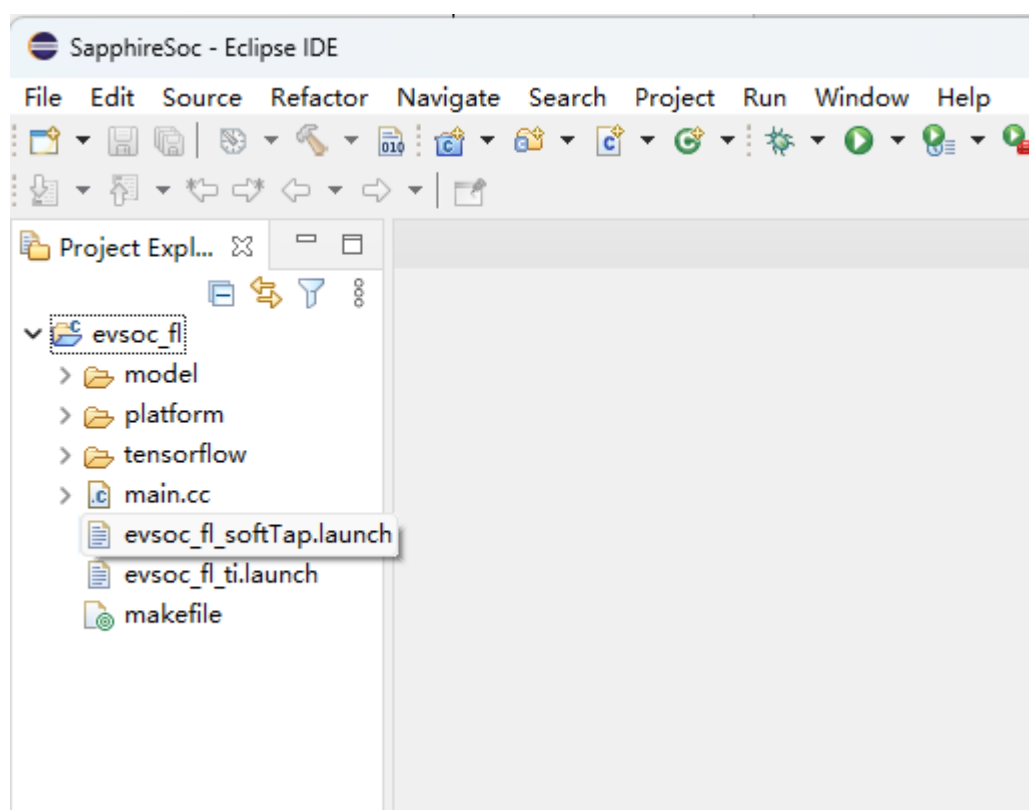
TinyML 工程开发框架基于 TensorFlow-Lite 进行开发，整体的软件编写步骤如下图，其中，蓝色部分的三个流程，只需重复执行单次，并且不同项目高度一致，填充数据和数据后处理，则会跟具体项目不同有较大区别。



TinyML 软件模板工程目录结构如下：

文件名	说明	备注
build	编译输出目录	
platform	Tinyml 驱动目录	
tensorflow	TFlite 库目录	
model	模型文件目录	
main.cc	工程 main 文件	
makefile	编译文件	

导入后的我工程，如图所示：



3.4.2 Tflite 模型导入

该不走进针对训练后量化的 tensorflow-lite 文件, 该文件类型的后缀名称为: model.tflite, 如目前没有 tflite 文件, 请参考 4.4.3 《训练后量化》章节。

3.4.2.1 配置运行环境

该脚本依赖 Efinity 工具中提供的 setup.bat (Linux 环境为 setup.sh), 打开命令行窗口并执行该命令:

```
命令提示符
C:\Users\chomp>d:
D:\>cd tinymml
D:\tinymml>dir
驱动器 D 中的卷是 Workspace
卷的序列号是 FAA2-2E0C

D:\tinymml 的目录
2022/11/07  17:33    <DIR>          .
2022/11/04  10:35    2,144,681 EVSOC_tinymml用户手册.docx
2022/11/07  17:32    <DIR>          face_landmark
2022/11/07  17:32    <DIR>          imgs
2022/10/25  16:14    1,810,045 quantization.zip
2022/11/07  17:32    <DIR>          tools
                2 个文件      3,954,726 字节
                4 个目录 150,714,155,008 可用字节

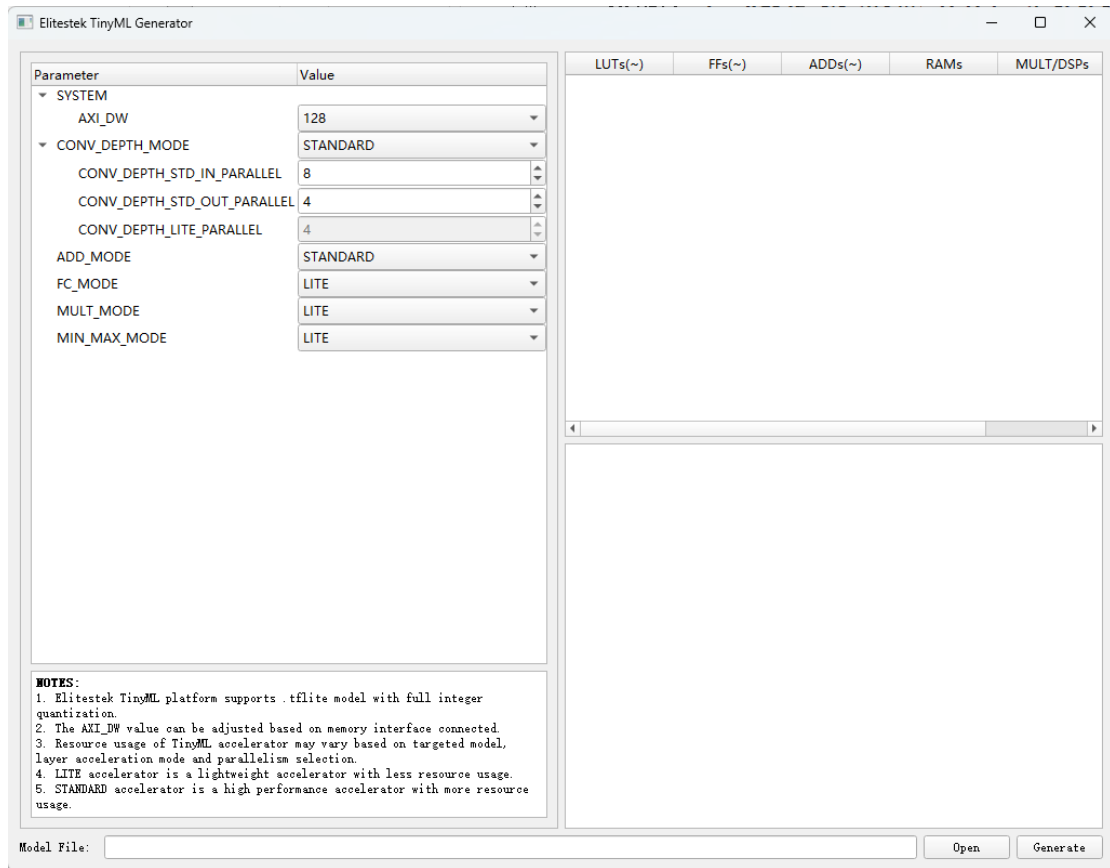
D:\tinymml>d:\Programs\Efinity\2022.1\bin\setup.bat
```

3.4.2.2 TinyML 参数配置

在刚才的命令行窗口，键入并执行：

python3 tools\tinymml_generator\tinymml_generator.py 命令

```
D:\tinymml>
D:\tinymml>
D:\tinymml>python3 tools\tinymml_generator\tinymml_generator.py
```

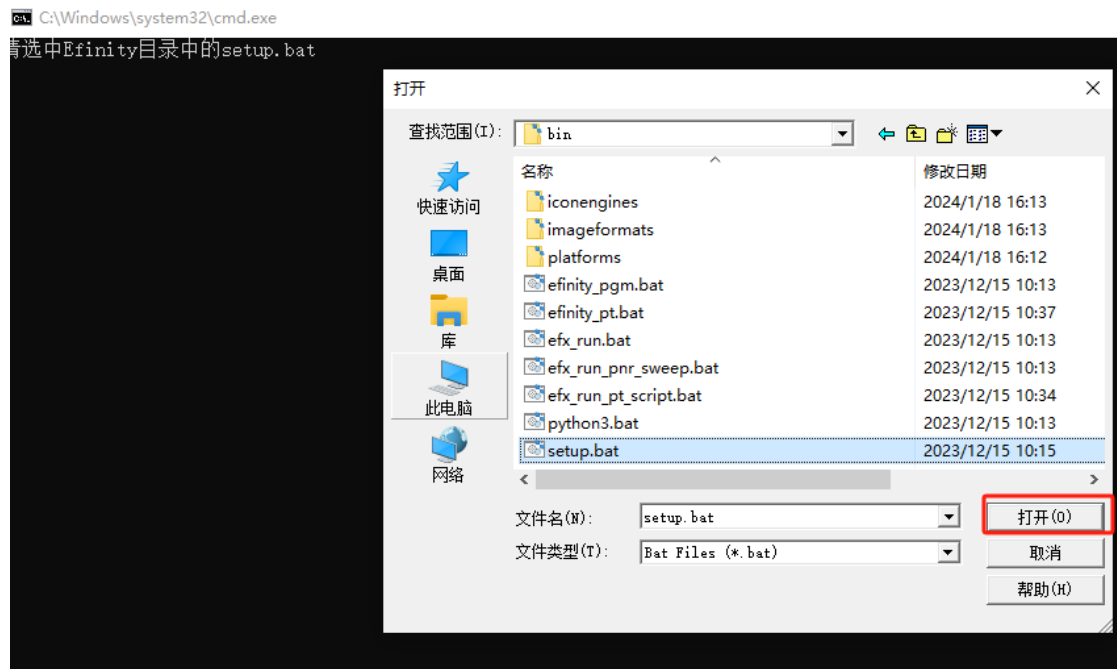


3.4.2.3 配置 TinyML 加速算子参数

TinyML 加速引擎提供了可配置参数，这些参数可以根据不同资源和性能，进行灵活配置，该工程路径/tools/tinymml_generator，提供 tinymml_generator 工具进行参数定制。可根据模型和可用资源大小进行调节，在资源占用和性能间选择最佳配置。参数配置步骤如下：

3.4.2.3.1 运行并配置工具

如下图所示，首次运行时，需要运行 run.bat 文件进行环境配置，运行后，需要选中 efinity 安装路径中的 setup.bat 文件（efinity\bin\setup.bat）进行配置。配置成功后，参数工具会自动运行 tinymml_generator。

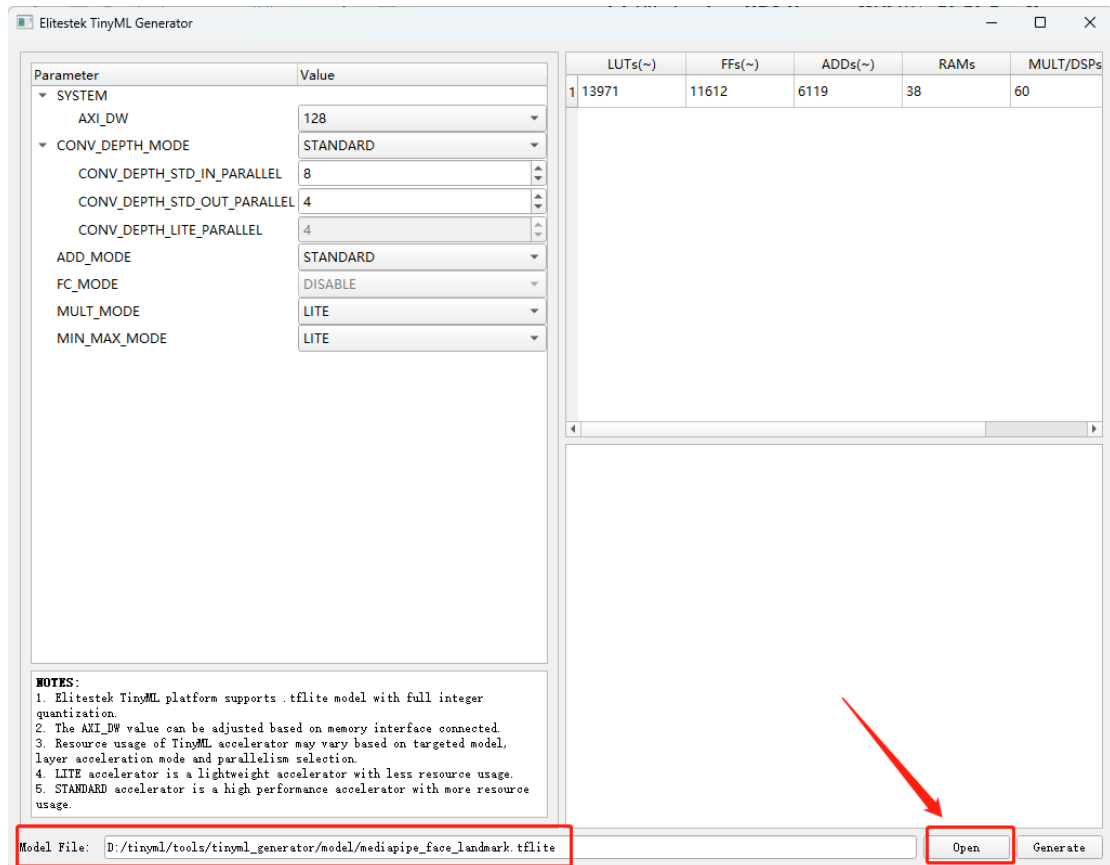


3.4.2.3.2 参数配置

工程文件夹 (tools/tinymml_generator/model) 中包含了如下几个模型的示例代码，分别对应关系为：

模型文件	Efinity 工程
mediapipe_face_landmark.tflite	Ti60F225_mediapipe_face_landmark_demo
mobilenetv1_person_detect.tflite	Ti60F225_mobilenetv1_person_detect_demo
yolo_person_detect.tflite	Ti60F225_yolo_person_detect_demo

我们选择 mediapipe_face_landmark.tflite 作为示例进行演示，首先点击 Open 按钮，选中并加载 Tensorflow-Lite 的 tflite 文件。



有关 TinyML 参数，这里有一份简单的说明：

参数	子参数	参数	默认值
AXI_DW	无	总线位宽	128
CONV_DEPTH_MODE		Standard: 标准加速	STANDARD
		Lite: 轻量加速	
		Disable: 关闭	
	CONV_DEPTH_STD_IN_PARALLEL	标准输入并行	4
	CONV_DEPTH_STD_OUT_PARALLEL	标准输出并行	4
	CONV_DEPTH_LITE_PARALLEL	轻量并行	4
	CONV_DEPTH_LITE_AW		7
ADD_MODE	无	STANDARD	STANDARD

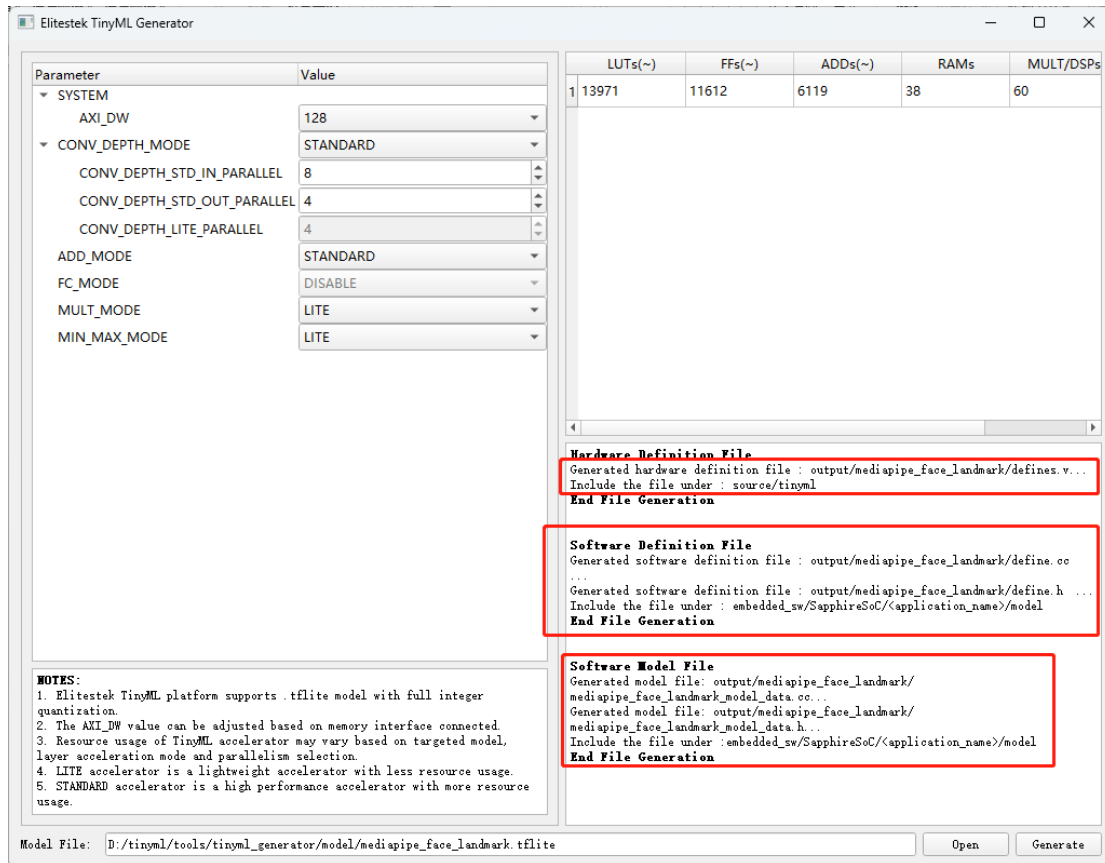
		LITE DISABLE	
FC_MODE		LITE	LITE
MULT_MODE		DISABLE	
MIN_MAX_MODE			
TINYML_CACHE		ENABLE DISABLE	ENABLE
	CATCH_DEPTH	缓存深度，更大的缓存可提供更快的加速	512

- Standard 模式：为高性能模式，资源占用较多
- Lite 模式：资源占用较低

3.4.2.4 生成模型参数文件

点击“Generate”按钮生成工程所需要的文件，如下图工具中会提示文件最终的生成路径，这些文件按照提示，放入指定位置：

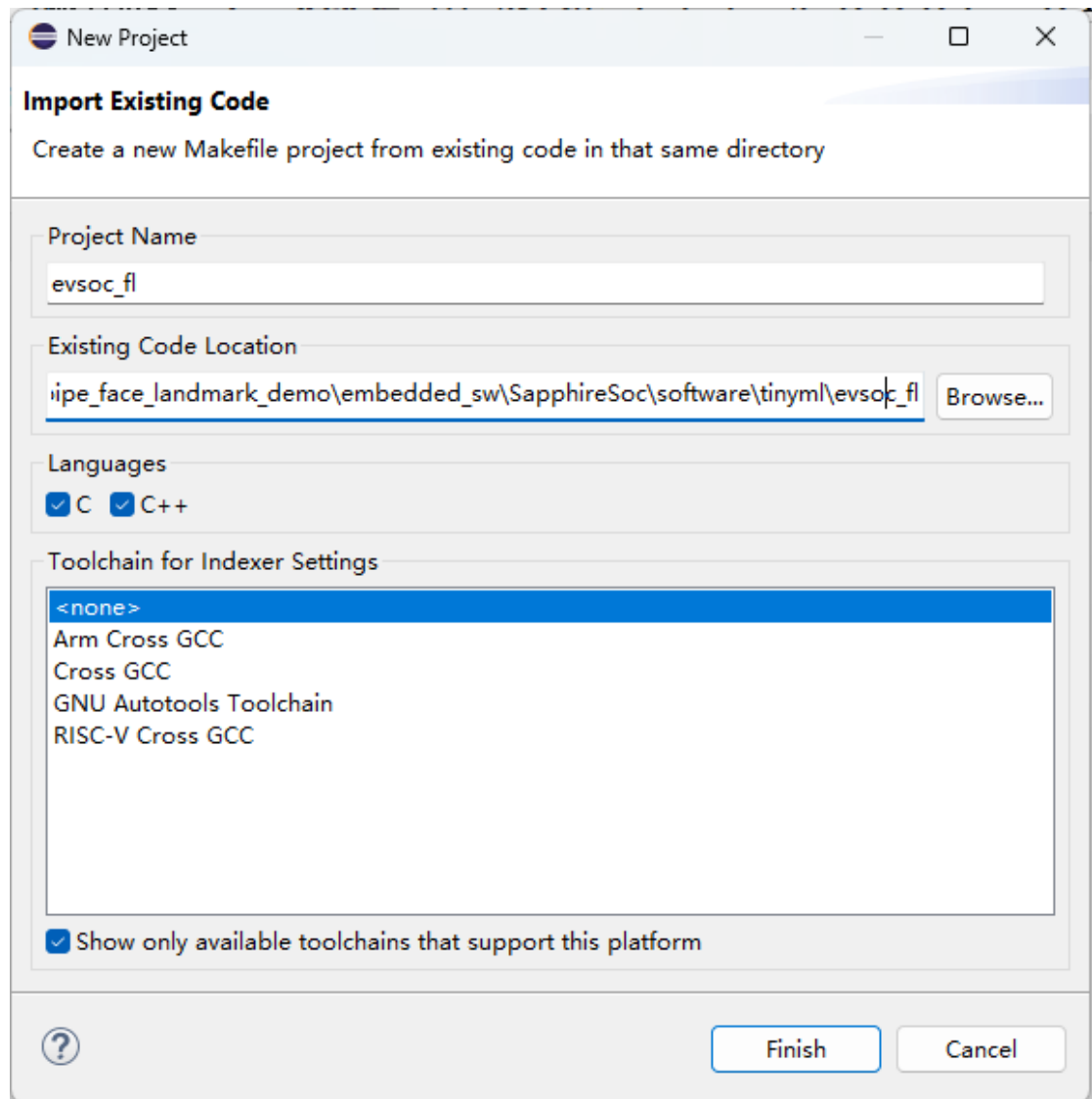
文件名	用途	存放路径
define.h define.cc	加速器软件参数	RISCV-SDK 工程目录文件夹： embedded_sw/SapphireSoC/\$project_name /model /
model_name_model_data.cc	导出的 tflite 模型文件	
model_name_model_data.h		
define.v	TinyML RTL硬件工程的参数	将该文件放入 rtl 工程目录的： \$project_name/source/tinyml



3.4.2.5 添加工程

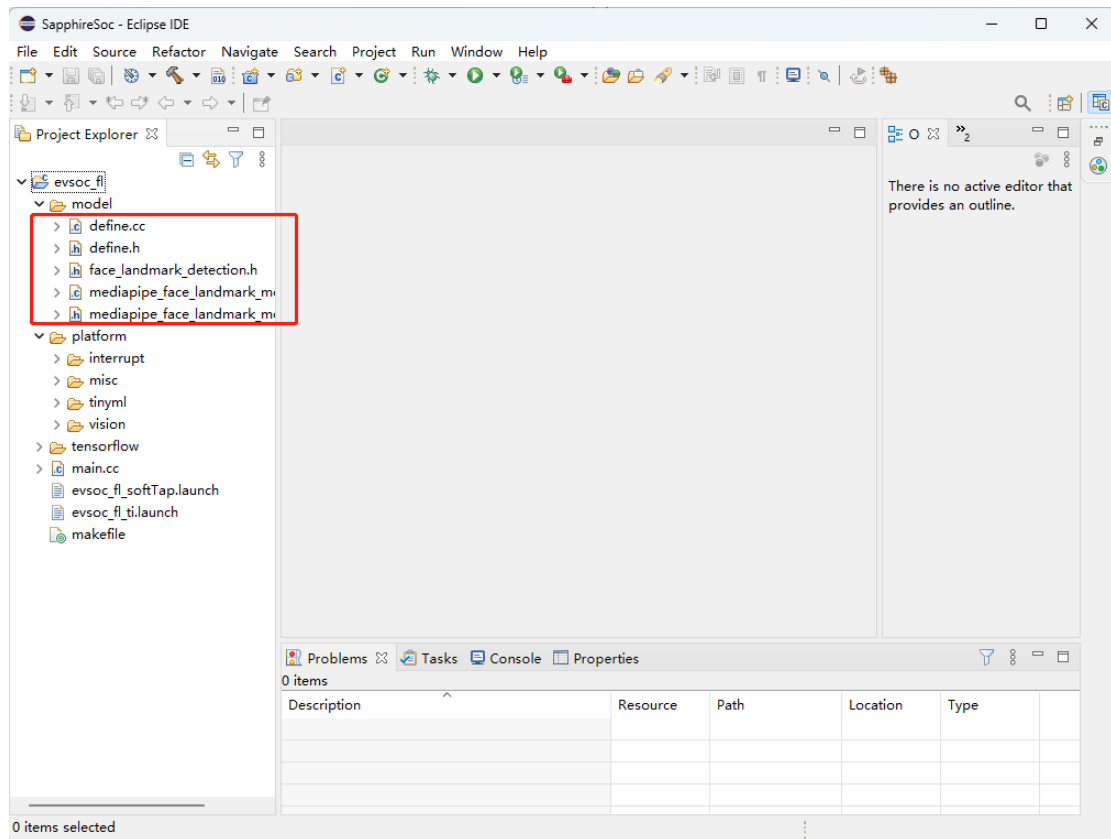
在导入软件工程前，需要进行工程导入和 workspace 切换，具体操作如下：

1. 打开 RISC-V-SDK，并选择 Sapphire 启动。
2. 点击 File -> switch Workspace -> Other。
3. 选中软件工程\$project/embedded_sw/ SapphireSoc
4. 选择导入 File -> New -> Makefile Project with Existing Code，选择内容如下，完成后点击 Finish。



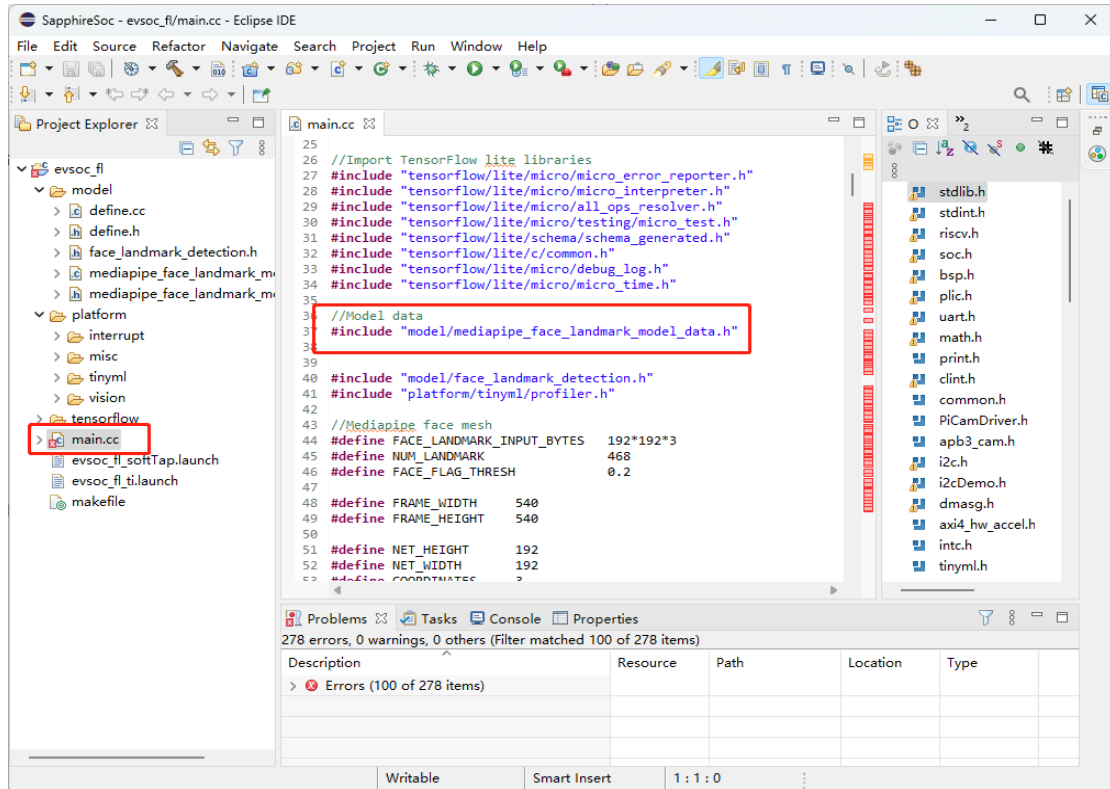
3.4.2.6 添加文件

将上述步骤生成的模型文件，放入工程目录的 model 文件夹中。



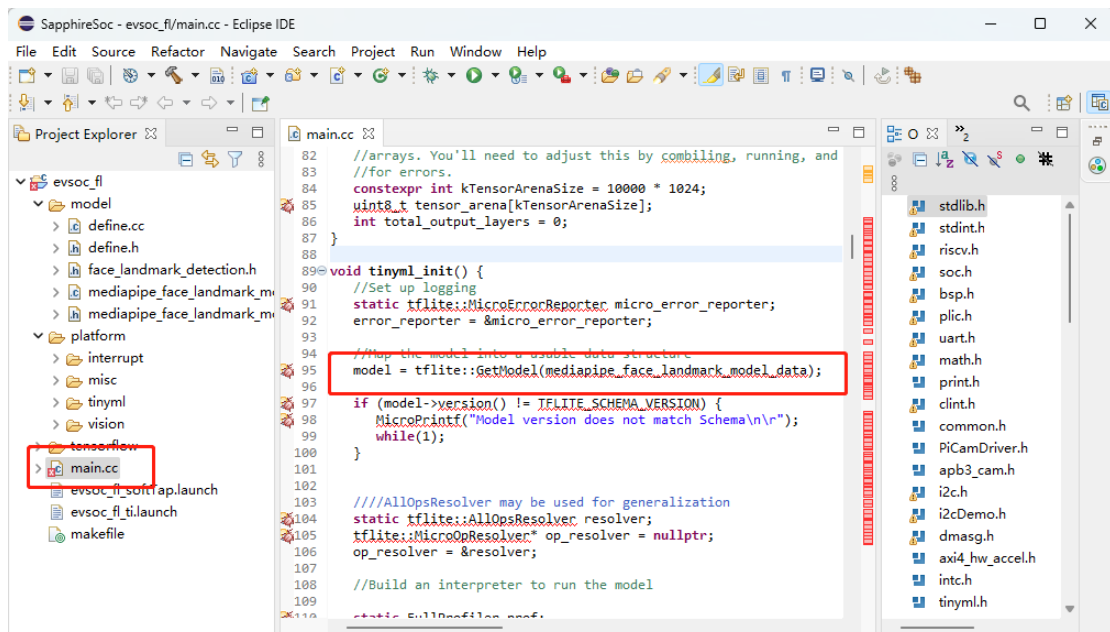
3.4.2.7 代码修改

将模型头文件 `face_landmark_model_data.h` 引用添加入 `main.cc`:

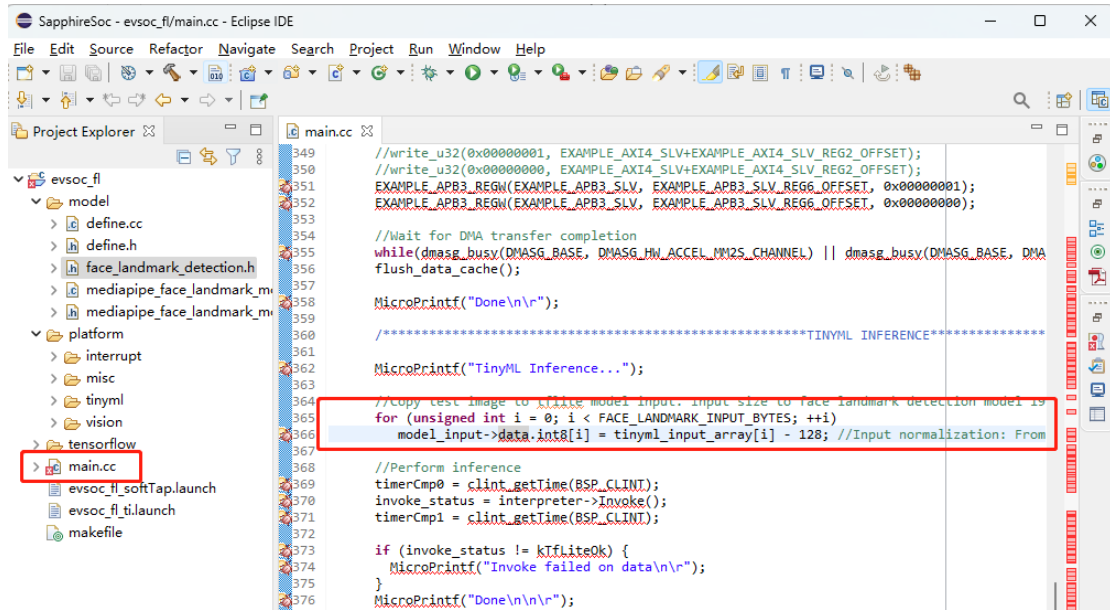


使用 tflite 库加载该模型数据，该代码为：

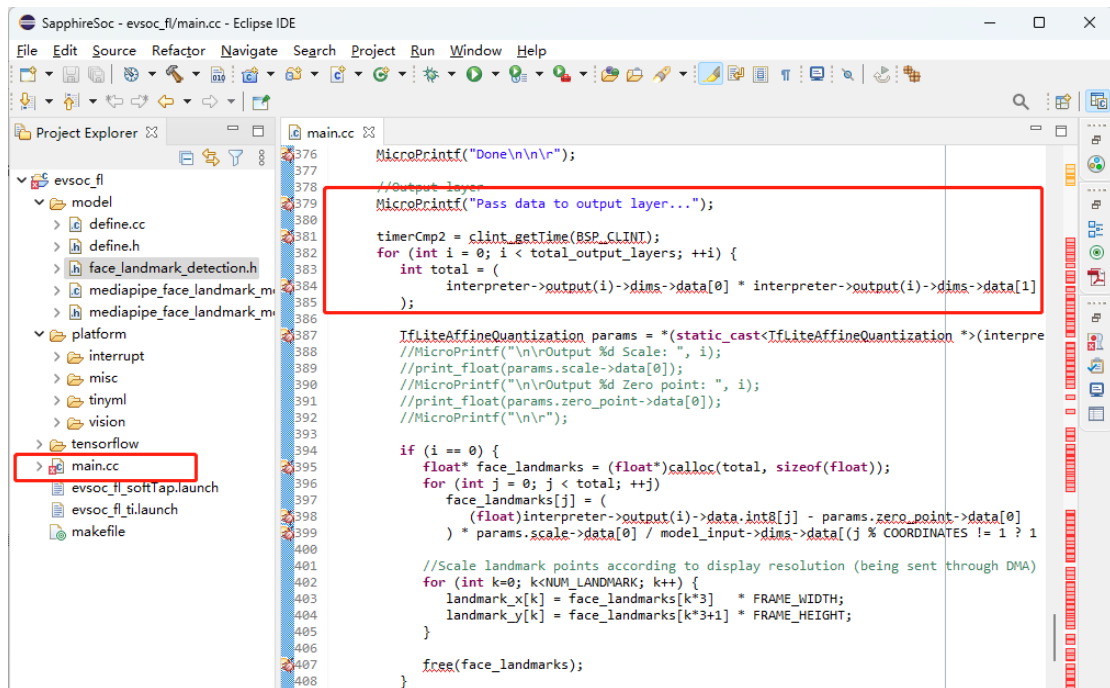
```
tflite::GetModel(mediapipe_face_landmark_model_data);
```



调用 tflite interpreter->Invoke() 函数，来启动模型推断。注意每次 Invoke 前，需要更新输入 Tensor，也就是 interpreter->input(0) 的数据，该项目使用 DMA 填充来自摄像头的的数据。

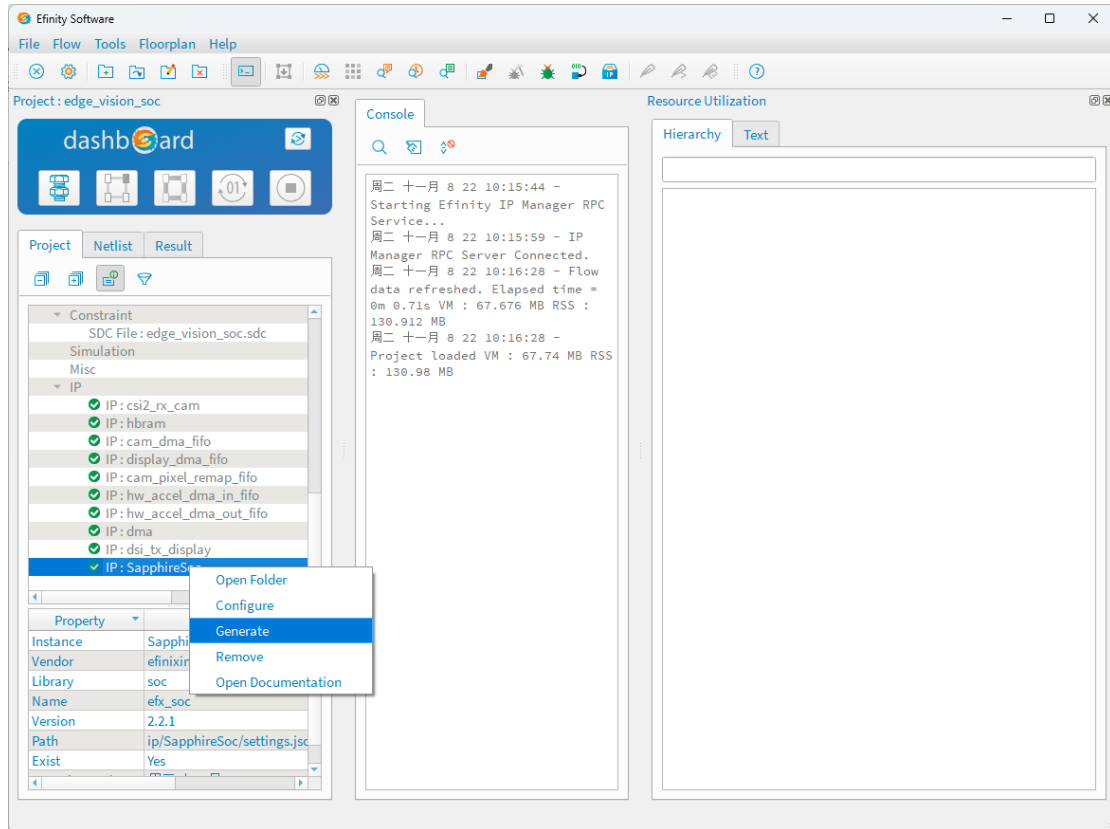


添加推理数据和推理代码之后，就是获取结果输出，Tensorflow-Lite 推理的结果保存在 interpreter->output 这个 Tensor 中，该 Tensor 的输出内容，与网络结构相关，通常如果只输出一组数据，那么结果就是 interpreter->output (0)。该例程中使用的 face_landmark 输出为面部点阵，数据稍复杂，如图所示：

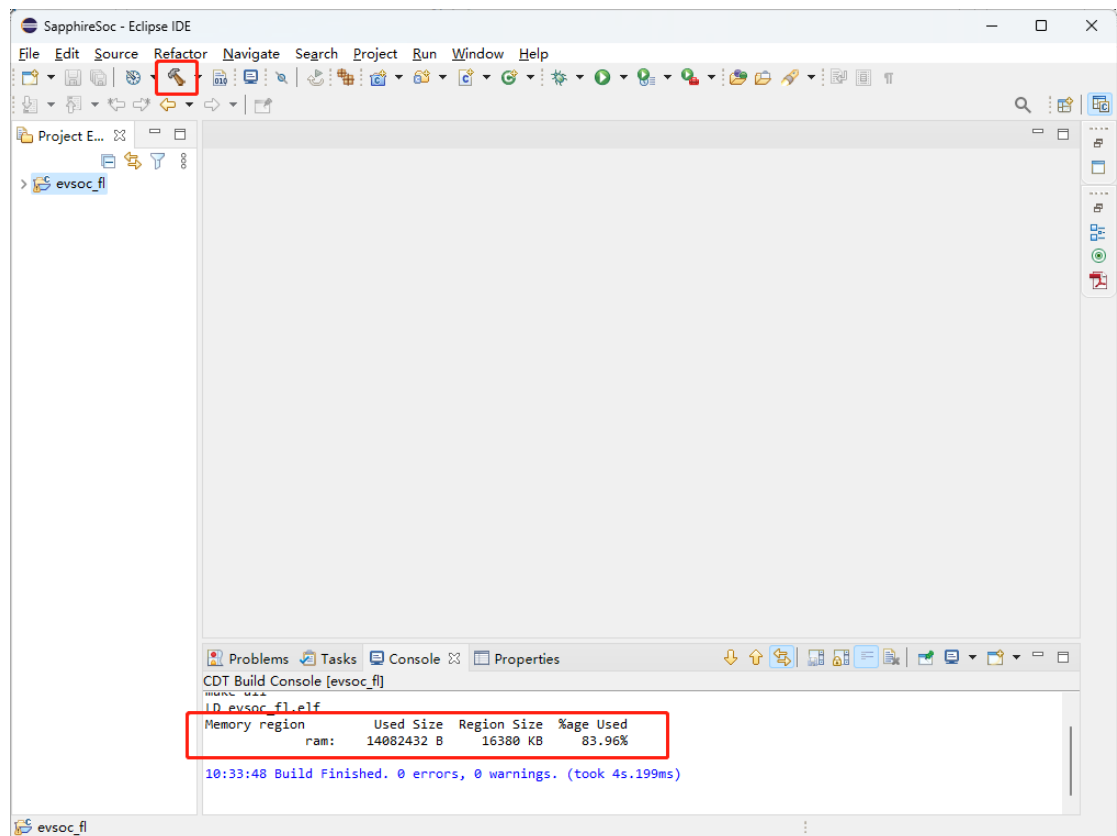


3.4.2.8 编译工程

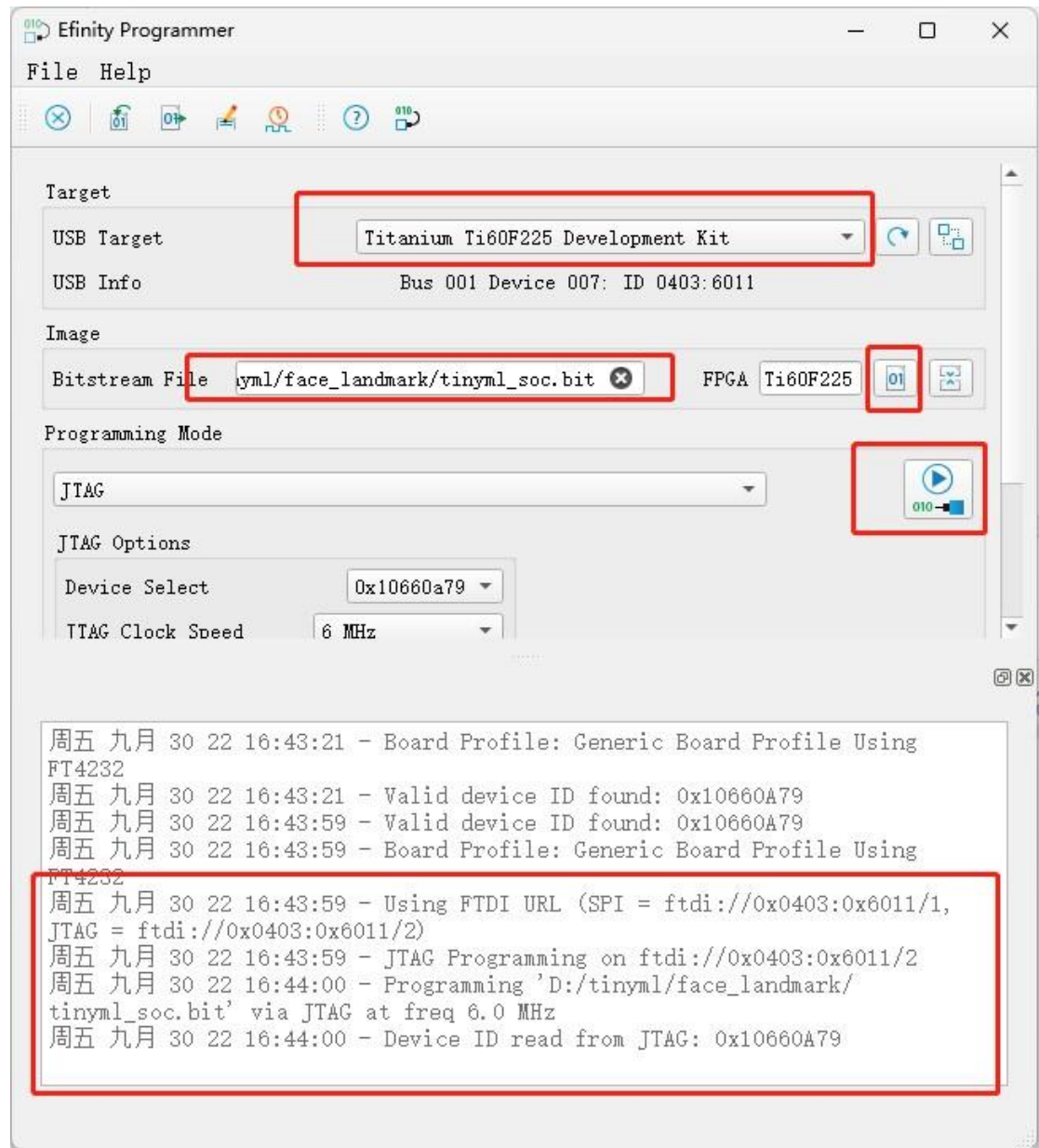
在编译软件工程之前，需要根据当前工程（tinyml_vision/Ti60F225_mediapipe_face_landmark_demo），生成 sapphire soc 的软件运行时，选中 ip，并选择 generate。



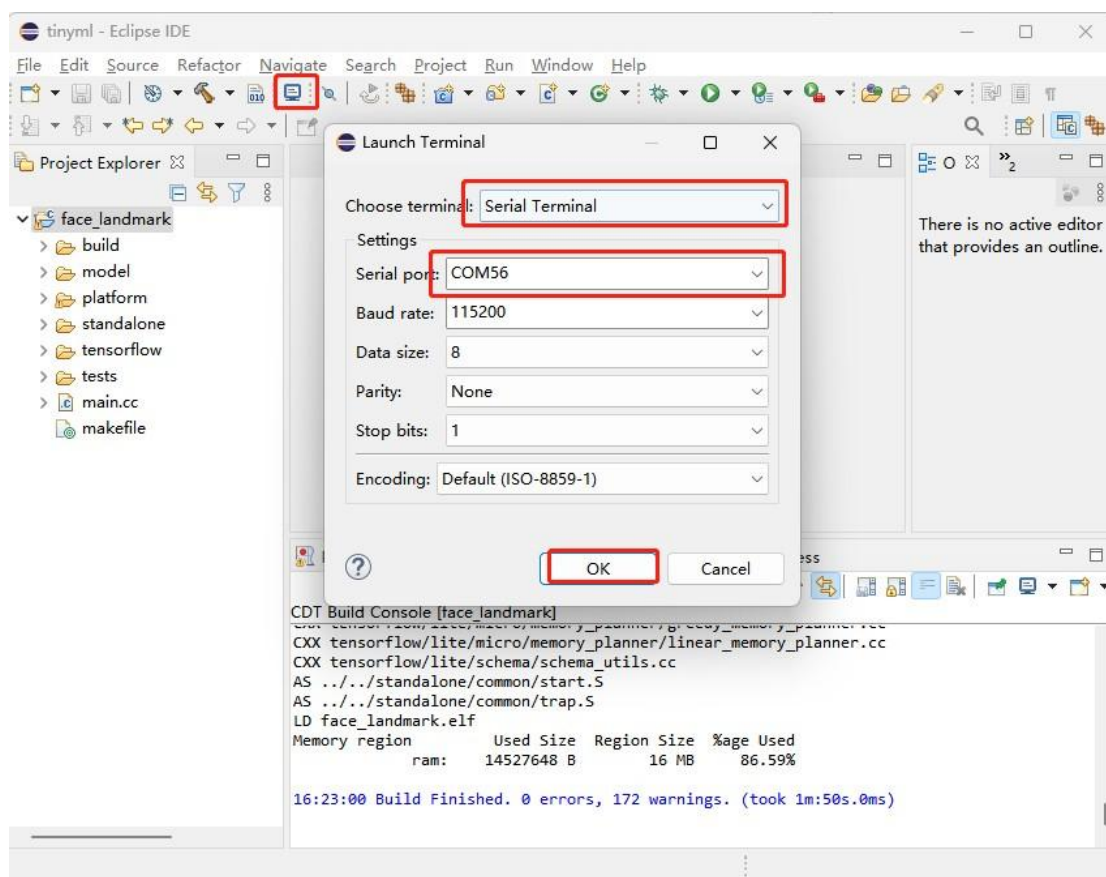
软件代码合并成功后，点击编译按钮编译该工程，编译完成后会输出该工程的内存占用大小，可根据实际情况判断是否需要减小 kTensorArenaSize 预分配空间的值，该值默认为（10M）。点击 build 按钮编译该工程：



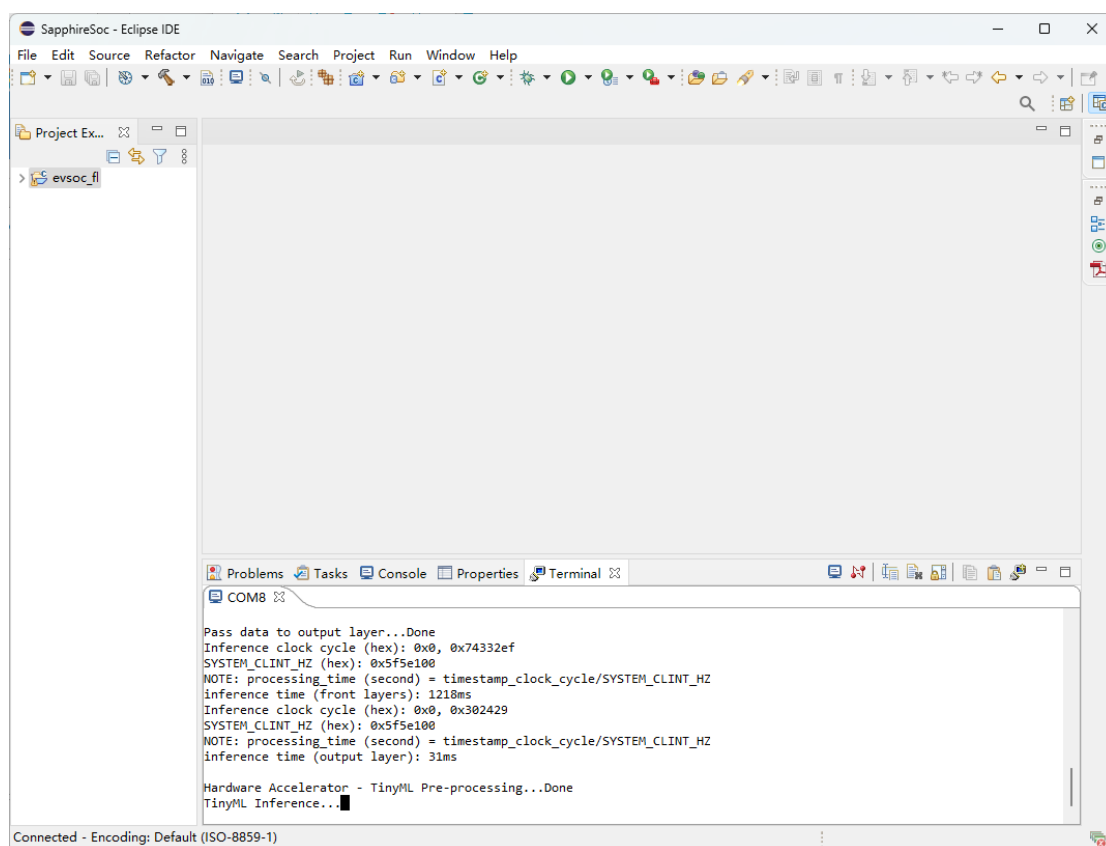
打开 Efinity Programmer，并烧写 bitstream:



bitstream 烧写成功后，需要在 RISC-V-SDK 中打开并配置终端，特别注意 COM 的名称会有所变动，Ti60F225 Devkit 有两个 COM 接口，选择第一个即可，下图仅作参考



运行该工程，等待大约 10 秒，会打印如下结果，可以看到一次推理耗时约为 940ms:



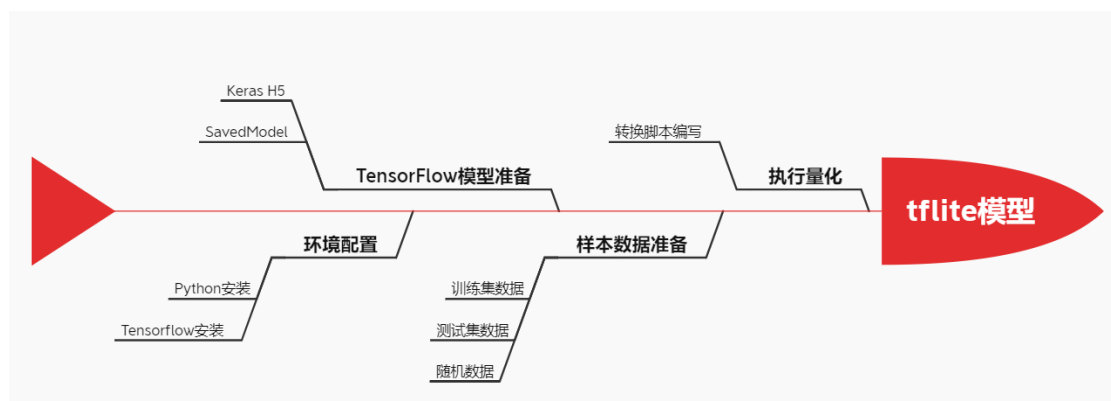
3.4.3 TensorFlow 训练后量化

3.4.3.1 概念

训练后量化的作用主要是为了节省计算资源和加速推断过程，在尽量不影响模型精度的前提下，提供更好的性能和更低的资源占用。实际的 AI 项目种，所使用的模型大多为 int8 量化模型，下图为一个性能参考：

技术	优势	硬件
动态范围	大小缩减至原来的四分之一，速度加快 2-3 倍	CPU
全整数	大小缩减至原来的四分之一，速度加快 3+ 倍	CPU、Edge TPU、
Float16 量化	大小缩减至原来的二分之一，速度加快 2 倍	CPU、GPU

- 全整数量化是将 FP32 $[\pm 2^{-126}, \pm (2^{128} - 2^{104})]$ 的取值范围，量化在 Int8 $[-128, 127]$ 范围内进行计算。将存储和带宽需求降低为原来的 1/4，计算能力提升的同时可节省大量资源。同样由于动态范围的缩小，在量化时就需要考虑裁剪/滑动取值窗口来进行参数校准。因此量化时，需要最接近实际场景的推理数据来对模型进行优化。
- 模型量化的整体流程为：

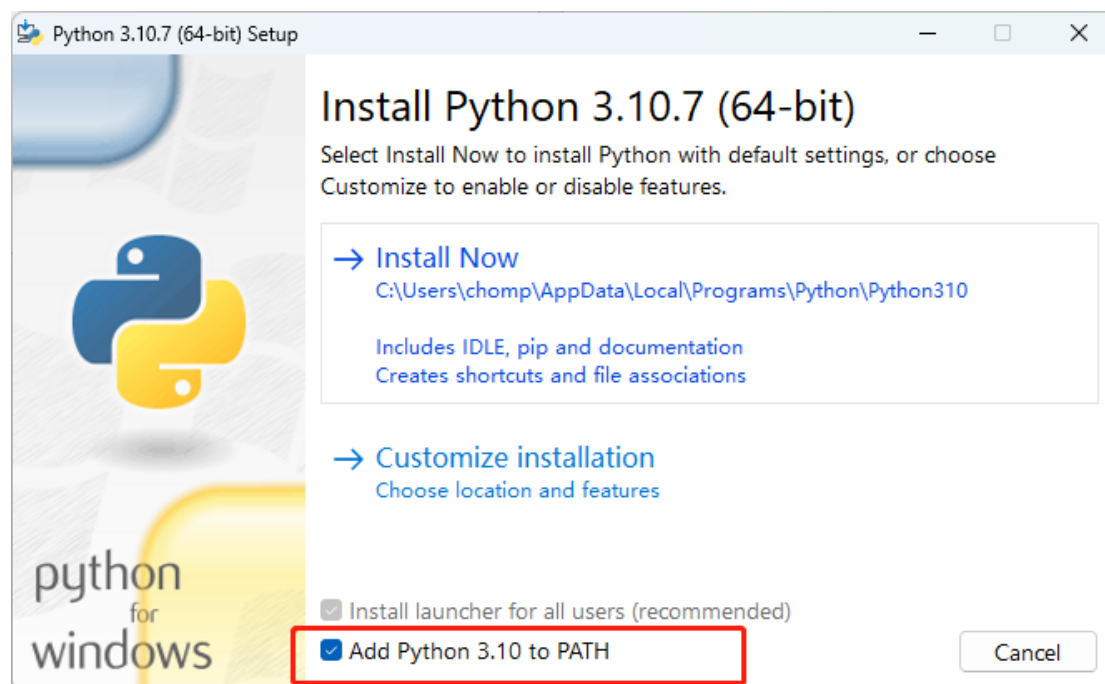


3.4.3.2 环境配置

Google 在 TensorFlow 库中，集成了 tflite 模型的导出工具，因此，在进行以下工作之前，首先需要安装 tensorflow 工具包。

Tensorflow 安装依赖 Python，首先需要下载安装 Python。

Windows 系统需下载安装包：[Python3.10.7](#)，另外如图所示，需要勾选 Add Python 3.10 to PATH 选项，用于命令行使用。



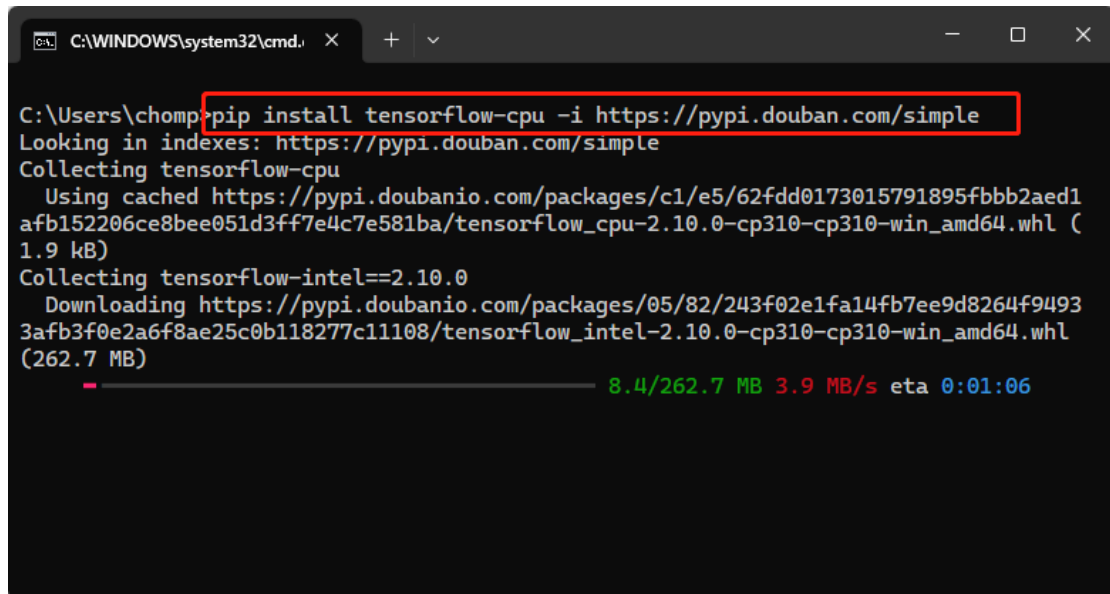
Linux 系统可直接使用命令行进行安装：

```
sudo apt-get install python3 python-is-python3
```

Python 安装完成后，即可使用命令行工具进行 tensorflow 安装，首先需要打开命令行窗口，Windows 系统下，为命令提示符，Linux 下则为终端模拟器，打开后键入如下命令：

```
pip install tensorflow-cpu -i https://pypi.douban.com/simple
```

Windows 命令提示符如图，等待安装完成即可。



```
C:\WINDOWS\system32\cmd...
C:\Users\chomp>pip install tensorflow-cpu -i https://pypi.douban.com/simple
Looking in indexes: https://pypi.douban.com/simple
Collecting tensorflow-cpu
  Using cached https://pypi.doubanio.com/packages/c1/e5/62fdd0173015791895fbbb2aed1afb152206ce8bee051d3ff7e4c7e581ba/tensorflow_cpu-2.10.0-cp310-cp310-win_amd64.whl (1.9 kB)
Collecting tensorflow-intel==2.10.0
  Downloading https://pypi.doubanio.com/packages/05/82/243f02e1fa14fb7ee9d8264f94933afb3f0e2a6f8ae25c0b118277c11108/tensorflow_intel-2.10.0-cp310-cp310-win_amd64.whl (262.7 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.4/262.7 MB 3.9 MB/s eta 0:01:06
```

3.4.3.3 TensorFlow 模型准备

TensorFlow2.0 目前支持常见的导出模型为 saved_model 和 keras h5 两种模型。前者通常为一个文件夹/压缩包。后者为.h5 结尾的单独文件，工程应用中，这两种模型文件都较为常见。

另外，在模型量化过程中，为了提升量化准度，也需要准备一些校准样本文件，校准文件是指对项目使用中的真实数据有代表性的数据，越符合实际分布越好。

如没有准备好校准数据，通常建议使用如 ImageNet、Cifar10、CoCo 等标准数据集中的数据。当然也可以直接生成随机数 tensor 进行填充，但是可能影响模型的精度。

3.4.3.4 编写量化脚本

Tensorflow 进行量化时，需要编写对应的 python 脚本，流程上比较固定，只是校准数据预取部分的函数需要根据情况自行实现，以下代码为范例代码，可在该脚本目录新建 imgs 文件夹，将校准图片放入该目录，然后通过命令行执行该脚本即可：

```
import tensorflow as tf
import numpy as np
import pathlib

# 该函数实现为整个模型转换过程中最关键的部分，需要根据实际情况提供量化时的校准数据。
# 理论上这里的数据应该接近实际数据分布越好，需求 100~500 份。
# 如果使用标准模型如 mnist, cifar10, imagenet, coco 等标准库进行模型训练，那么可以直接使用 test 标签下的数据。
# 如果为自定义图片，则需要根据场景提供合适的图片集
```

```

# 将图片加载放入 train_images 变量中
# def representative_dataset(): #需要根据模型数据构造编写
#     #从训练数据中抽取 100 份，实现参考数据输入
#     data = tf.data.Dataset.from_tensor_slices(train_images).batch(1).take(100)
#     for tensor in data:
#         yield [tensor]
img_height = 0
img_width = 0
def representative_dataset():
    data_dir = pathlib.Path("imgs").glob("*.jpg")
    for i in data_dir:
        img = tf.io.read_file(i.as_posix())
        tensor = tf.io.decode_image(img, channels=3, dtype=tf.dtypes.float32)
        tensor = tf.image.resize(tensor, [img_width, img_height])
        yield [tf.expand_dims(tensor, 0)]

#训练后的 FP32 模型加载
model_name = 'trained_models' #saved_model 文件格式
# model_name= 'trained_models.h5' #Keras H5 文件格式

model = tf.keras.models.load_model(model_name)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
[_, img_width, img_height, _] = model.layers[0].input_shape

converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_quant_model = converter.convert()
open(model_name + ".tflite", "wb").write(tflite_quant_model)

```

3.4.3.5 实例参考（Resnet）

附件文件夹中 quantization.zip 压缩包，提供了该实例所需要用到的文件目录，该实例使用一个已经训练好的 Resnet 模型进行训练后量化，文件内容为：

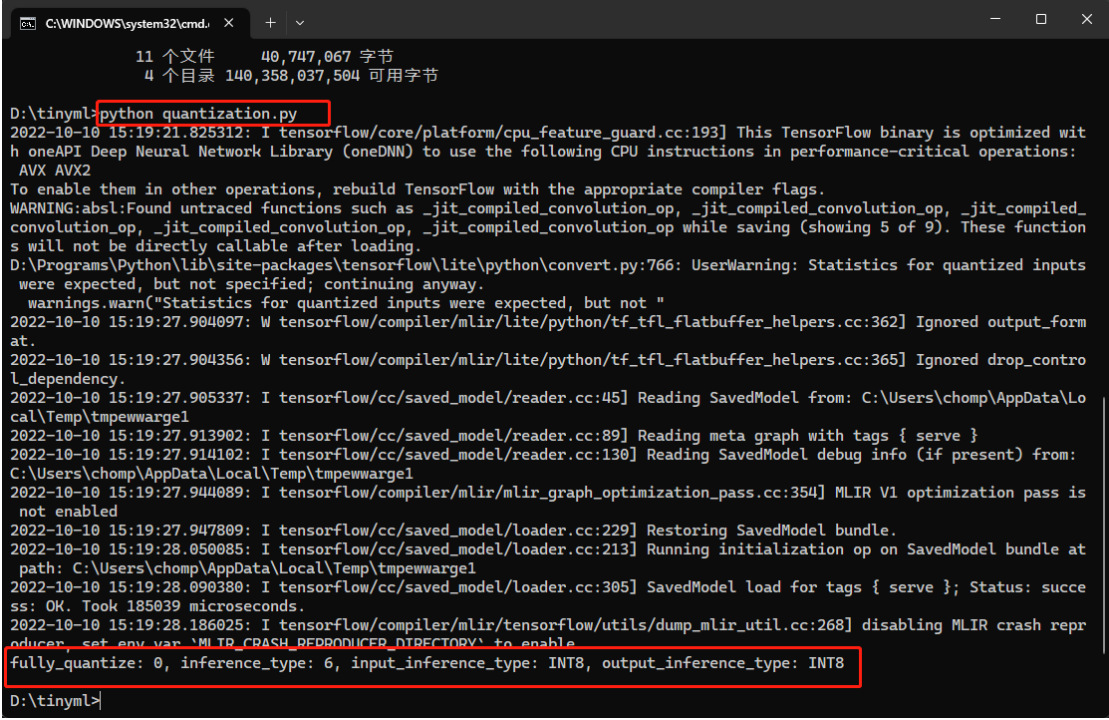
文件类型	描述	示例文件
模型文件	TensorFlow 所支持的两种模型格式, save_model 和 H5 格式	trained_models
		trained_model.h5

参考数据集	用于 int8 量化时的参数矫正，提高模型准确率。来源为模型训练时的训练集、测试集、表达集。	Imgs/*.jpg
量化脚本	简单模型转换代码	quantization.py
Tflite 模型	量化后自动生成	trained_models.tflite

附件文件进行解压，并使用命令行执行如下命：

```
python quantization.py
```

执行结果：



```

C:\WINDOWS\system32\cmd.exe
11 个文件      40,747,067 字节
4 个目录      140,358,037,504 可用字节

D:\tinyml>python quantization.py
2022-10-10 15:19:21.825312: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with
oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations:
AVX AVX2
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Found untraced functions such as _jit_compiled_convolution_op, _jit_compiled_convolution_op, _jit_compiled_
convolution_op, _jit_compiled_convolution_op, _jit_compiled_convolution_op while saving (showing 5 of 9). These function
s will not be directly callable after loading.
D:\Programs\Python\lib\site-packages\tensorflow\lite\python\convert.py:766: UserWarning: Statistics for quantized inputs
were expected, but not specified; continuing anyway.
  warnings.warn("Statistics for quantized inputs were expected, but not ")
2022-10-10 15:19:27.904097: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:362] Ignored output_form
at.
2022-10-10 15:19:27.904356: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:365] Ignored drop_contro
l_dependency.
2022-10-10 15:19:27.905337: I tensorflow/cc/saved_model/reader.cc:45] Reading SavedModel from: C:\Users\chomp\AppData\Lo
cal\Temp\tmpewwargel
2022-10-10 15:19:27.913902: I tensorflow/cc/saved_model/reader.cc:89] Reading meta graph with tags { serve }
2022-10-10 15:19:27.914102: I tensorflow/cc/saved_model/reader.cc:130] Reading SavedModel debug info (if present) from:
C:\Users\chomp\AppData\Local\Temp\tmpewwargel
2022-10-10 15:19:27.944089: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:354] MLIR V1 optimization pass is
not enabled
2022-10-10 15:19:27.947809: I tensorflow/cc/saved_model/loader.cc:229] Restoring SavedModel bundle.
2022-10-10 15:19:28.050085: I tensorflow/cc/saved_model/loader.cc:213] Running initialization op on SavedModel bundle at
path: C:\Users\chomp\AppData\Local\Temp\tmpewwargel
2022-10-10 15:19:28.090380: I tensorflow/cc/saved_model/loader.cc:305] SavedModel load for tags { serve }; Status: succe
ss: OK. Took 185039 microseconds.
2022-10-10 15:19:28.186025: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:268] disabling MLIR crash repr
oducer, set env var 'MLIR_CRASH_REPRODUCER_DIRECTORY' to enable
fully_quantize: 0, inference_type: 6, input_inference_type: INT8, output_inference_type: INT8
D:\tinyml>

```

最终在同目录下，生成 trained_model.tflite 文件。可参考 4.4.2《创建 TinyML 工程》使用该模型文件。